Noctalia

Launcher Provider

Launcher providers extend the Noctalia launcher with custom search sources, command handlers, and browsable content. They allow plugins to add new functionality to the app launcher.

Overview

A launcher provider can:

- Add custom search results to the launcher
- Handle commands (e.g., >kaomoji, >todo)
- Provide category-based browsing
- Support auto-paste for quick input

Basic Structure

Here's a minimal launcher provider:

```
import QtQuick
import qs.Commons

Item {
  id: root

  // Required properties
  property var pluginApi: null
  property var launcher: null
  property string name: "My Provider"

  // Check if this provider handles the command
  function handleCommand(searchText) {
    return searchText.startsWith(">mycommand")
  }

  // Return available commands when user types ">"
  function commands() {
    return [{
      "name": ">mycommand",
      "description": "Search my custom content",
      "icon": "search",
      "isTablerIcon": true,
      "onActivate": function() {
        launcher.setSearchText(">mycommand ")
      }
    }]
  }

  // Get search results
  function getResults(searchText) {
    if (!searchText.startsWith(">mycommand")) {
      return []
    }

    var query = searchText.slice(10).trim() // Remove ">mycommand "
    // Return results based on query
    return [{
      "name": "Result 1",
```

```
    "description": "A sample result",
    "icon": "star",
```



```
      // Do something when activated
      launcher.close()
    }
  }]
 }
}
```

Manifest Entry Point

Add the launcherProvider entry point to your manifest.json:

```
{
 "id": "my-provider",
 "name": "My Provider",
 "version": "1.0.0",
 "author": "Your Name",
 "description": "A custom launcher provider",
 "minNoctaliaVersion": "3.9.0",
 "entryPoints": {
  "launcherProvider": "LauncherProvider.qml"
 }
}
```

> **Note**
>
> Launcher providers require Noctalia Shell version 3.9.0 or later.

Provider Properties

Required Properties

Property	Type	Description

launcher	var	Reference to the Launcher panel (injected)
name	string	Display name for the provider

Optional Properties

Property	Type	Default	Description
handleSearch	bool	false	Participate in regular search (not just commands)
supportedLayouts	string	"both"	Layout support: "both", "list", or "grid"
preferredGridColumns	int	5	Number of columns in grid view. This is then scaled based on the density setting.
preferredGridCellRatio	real	1.0	Cell aspect ratio (width/height)
supportsAutoPaste	bool	false	Enable auto-paste feature
categories	var	[]	Array of category IDs for browsing
showsCategories	bool	false	Show category chips in browse mode
categoryIcons	var	{}	Object mapping category IDs to icon names
selectedCategory	string	""	Currently selected category
emptyBrowsingMessage	string	""	Message when category has no items
ignoreDensity	bool	true	Whether to use the density configured in the global launcher settings. If density is ignored, “comfortable” density is used instead.

Provider Functions

Called when the provider is registered. Use this to load data or perform initialization:

```
function init() {
  Logger.i("MyProvider", "Initialized")
  // Load database, initialize state, etc.
}
```

onOpened()

Called each time the launcher panel opens:

```
function onOpened() {
  selectedCategory = "all" // Reset to default category
}
```

handleCommand(searchText)

Check if this provider should handle the current input. Return `true` to indicate you handle this command:

```
function handleCommand(searchText) {
  return searchText.startsWith(">emoji") ||
         searchText.startsWith(">kaomoji")
}
```

commands()

Return an array of available commands when user types `>`:

```
function commands() {
  return [
    "name": ">emoji",
    "description": "Search and insert emoji",
    "icon": "mood-smile",
    "isTablerIcon": true,
    "onActivate": function() {
      launcher.setSearchText(">emoji ")
    }
   }
  ]
}
```


getResults(searchText)

Return an array of result objects based on the search text:

```
function getResults(searchText) {
  if (!searchText.startsWith(">emoji")) {
    return []
  }

  var query = searchText.slice(6).trim()

  if (query === "") {
    // Browse mode - show categories or all items
    return getAllEmojis()
  } else {
    // Search mode - filter by query
    return searchEmojis(query)
  }
}
```

selectCategory(category)

Handle category selection in browse mode:

```
function selectCategory(category) {
  selectedCategory = category
  launcher.updateResults()
 }
}
```


getCategoryName(category)

Return a localized display name for a category:

```
function getCategoryName(category) {
  const names = {
    "all": "All",
    "recent": "Recent",
    "favorites": "Favorites"
  }
  return names[category] || category
}
```

Result Object Structure

Each result returned by getResults() should have this structure:

```
{
  // Display
  "description": "Subtitle text",   // Secondary text (optional)

  // Icon options (choose one)
  "icon": "star",                // Icon name
  "isTablerIcon": true,          // Use Tabler icon set
  "isImage": false,              // Is this an image?
  "displayString": "🎉",         // Text to show instead of icon (for emoji)
  "hideIcon": false,             // Hide the icon entirely

  // Layout
  "singleLine": false,           // Clip to single line height

  // Auto-paste support
  "autoPasteText": "🎉",         // Text to paste when auto-paste enabled

  // Reference
  "provider": root,              // Reference to provider (for actions)

  // Callbacks
  "onActivate": function() {     // Called when result is selected
    // Copy to clipboard, open URL, etc.
    launcher.close()
  },
  "onAutoPaste": function() {    // Called before auto-pasting
    // Record usage, update history, etc.
  }
}
```

Category-Based Browsing

For providers with categories, set up the category system:

```
Item {
  id: root
  property var pluginApi: null
  property var launcher: null
  property string name: "Emoji"

  property bool showsCategories: true
  property string selectedCategory: "recent"
  property var categories: ["recent", "people", "animals", "food", "travel"]

  property var categoryIcons: ({
    "recent": "clock",
    "people": "user",
    "animals": "paw",
    "food": "apple",
    "travel": "plane"
  })

  function getCategoryName(category) {
    const names = {
      "recent": "Recent",
      "people": "People",
      "animals": "Animals",
      "food": "Food",
      "travel": "Travel"
    }
    return names[category] || category
  }

  function selectCategory(category) {
    selectedCategory = category
    if (launcher) {
      launcher.updateResults()
    }
  }

  function getResults(searchText) {
    // Filter by selectedCategory when browsing
```

```
    if (searchText === ">emoji " || searchText === ">emoji") {
      showsCategories = true
```



```
    // Hide categories when searching
    showsCategories = false
    return searchEmojis(searchText.slice(6).trim())
  }
}
```

Complete Example: Kaomoji Provider

Here's a real-world example from the kaomoji-provider plugin:

```
import QtQuick
import Quickshell
import qs.Commons

Item {
  id: root

  property var pluginApi: null
  property string name: "Kaomoji"
  property var launcher: null
  property bool handleSearch: false
  property string supportedLayouts: "list"
  property bool supportsAutoPaste: true

  property string selectedCategory: "all"
  property var database: ({})
  property bool loaded: false

  property var categories: [
    "all", "smiling", "heart", "sad", "angry", "surprised"
  ]

  property var categoryIcons: ({
    "all": "list",
    "smiling": "mood-smile",
    "heart": "heart",
    "sad": "mood-sad",
    "angry": "mood-angry",
    "surprised": "mood-surprised"
  })

  function getCategoryName(category) {
    const names = {
      "all": "All",
      "smiling": "Happy",
      "heart": "Love",
      "sad": "Sad",
      "angry": "Angry",
```

```
  "surprised": "Surprised"
 }
```



```
function init() {
 if (pluginApi && pluginApi.pluginDir && !loaded) {
  databaseLoader.path = pluginApi.pluginDir + "/database.json"
 }
}

FileView {
 id: databaseLoader
 path: ""
 watchChanges: false

 onLoaded: {
  try {
   root.database = JSON.parse(text())
   root.loaded = true
   if (root.launcher) {
    root.launcher.updateResults()
   }
  } catch (e) {
   Logger.e("KaomojiProvider", "Failed to parse database:", e)
  }
 }
}

function selectCategory(category) {
 selectedCategory = category
 if (launcher) {
  launcher.updateResults()
 }
}

function onOpened() {
 selectedCategory = "all"
}
```

```
function handleCommand(searchText) {
  return searchText.startsWith(">kaomoji")
```



```
function commands() {
  return [{
    "name": ">kaomoji",
    "description": "Browse and search kaomoji",
    "icon": "mood-wink",
    "isTablerIcon": true,
    "onActivate": function() {
      launcher.setSearchText(">kaomoji ")
    }
  }]
}

function getResults(searchText) {
  if (!searchText.startsWith(">kaomoji")) {
    return []
  }

  if (!loaded) {
    return [{
      "name": "Loading...",
      "description": "Loading kaomoji database...",
      "icon": "refresh",
      "isTablerIcon": true,
      "onActivate": function() {}
    }]
  }

  var query = searchText.slice(8).trim().toLowerCase()
  var results = []

  if (query === "") {
    // Browse mode
    var keys = Object.keys(database)
    var filtered = selectedCategory === "all"
      ? keys
      : keys.filter(function(k) {
```

```
        var tags = database[k].tags || []
        return tags.indexOf(selectedCategory) !== -1
```



```
    for (var i = 0; i < Math.min(filtered.length, 100); i++) {
      results.push(formatEntry(filtered[i], database[filtered[i]]))
    }
  } else {
    // Search mode
    var keys = Object.keys(database)
    for (var i = 0; i < keys.length && results.length < 50; i++) {
      var kaomoji = keys[i]
      var entry = database[kaomoji]
      var tags = (entry.tags || []).join(" ").toLowerCase()
      if (tags.indexOf(query) !== -1) {
        results.push(formatEntry(kaomoji, entry))
      }
    }
  }

  return results
}

function formatEntry(kaomoji, entry) {
  return {
    "name": kaomoji,
    "description": (entry.tags || []).slice(0, 5).join(", "),
    "hideIcon": true,
    "singleLine": true,
    "onActivate": function() {
      var escaped = kaomoji.replace(/'/g, "'\\''")
      Quickshell.execDetached([
        "sh", "-c",
        "printf '%s' '" + escaped + "' | wl-copy"
      ])
      launcher.close()
    }
  }
}
```

```
}
```


IPC Integration

Launcher providers can be toggled via IPC using the pluginApi.toggleLauncher() method. This allows users to bind keyboard shortcuts to open your provider directly.

Setting Up IPC

Add an IPC handler in your Main.qml:

```
import QtQuick
import Quickshell.Io

Item {
  property var pluginApi: null

  IpcHandler {
    target: "plugin:my-provider"

    function toggle() {
      if (!pluginApi) return;
      pluginApi.withCurrentScreen(screen => {
        pluginApi.toggleLauncher(screen);
      });
    }
  }
}
```

Command Prefix

Set the commandPrefix in your manifest.json metadata to define the launcher command:

```
{
  "metadata": {
  }
}
```


When toggleLauncher() is called, it opens the launcher with >mycommand pre-filled. If not specified, the plugin ID is used as the prefix.

Usage

```
# Toggle the launcher with your provider's prefix
qs -c noctalia-shell ipc call plugin:my-provider toggle
```

This can be bound to a keyboard shortcut in your compositor configuration for quick access.

> **Tip**
>
> The toggleLauncher method works correctly in both normal and overlay launcher modes, unlike direct panel access which only works in normal mode.

Best Practices

1. **Use commands for discoverability** - Register commands so users can find your provider by typing >
2. **Handle loading states** - Show loading indicators when fetching data
3. **Limit results** - Return at most 50-100 results to keep the UI responsive
4. **Support categories for large datasets** - Use categories to organize many items
5. **Provide clear descriptions** - Help users understand what each result does
6. **Close the launcher** - Call launcher.close() after the user makes a selection
7. **Use appropriate layouts** - Set supportedLayouts based on your content (grid for visual items, list for text)

See Also

- Control Center Widget - Quick action buttons
- Plugin API - Full API reference
- Manifest Reference - Plugin configuration